● AUDITORÍA & ESPECIFICACIÓN TÉCNICA DE SOFTWARE

JUEGO

Documento Maestro de Arquitectura, Endpoints, Modelos de Dominio y Métricas de Calidad

Proyecto:	juego
Lenguaje Dominante:	JavaScript
Score de Calidad:	**71 / 100 (Aceptable)**
Total Archivos Analizados:	21
Fecha de Generación:	16/10/2026 02:11 UTC
Motor de Generación:	AutoDocs AI Enterprise (Groq IA + PlantUML/Kroki)

21	42	0	3
ARCHIVOS	FUNCIONES	CLASES	ENDPOINTS

Tabla de Contenido

Índice navegable de capítulos, secciones técnicas y diagramas arquitectónicos.

Tabla de Contenido 2

Documentación del Proyecto 5

Resumen Ejecutivo . . . 5

Stack Tecnológico . . . 5

Estadísticas del Proyecto . . . 5

2. Documentación de Arquitectura y Diseño . . . 6

2.1 Estructura de Directorios . . . 6

2.2 Distribución por Capas . . . 6

2.3 Distribución de Lenguajes . . . 6

2.4 Diagrama de Infraestructura . . . 7

2.5 Diagrama de Casos de Uso . . . 7

2.6 Diagrama de Clases . . . 8

2.7 Patrones Detectados . . . 8

2.8 Insights de Arquitectura (IA) . . . 8

3. Documentación Técnica de Código . . . 8

3.1 Guía de Configuración del Entorno de Desarrollo . . . 8

3.2 Documentación de API . . . 9

1. ¿Cuál es su propósito en lenguaje simple? . . . 10

2. ¿Qué hace este endpoint? . . . 10

3.3 Estado de Comentarios y Docstrings . . . 11

3.4 Planes de Prueba (Testing) . . . 11

4. Clases . . . 11

5. Documentación de Funciones 11

Módulo: `backend/server.js` *11*

Módulo: `backend/test.js` *17*

Módulo: `backend/words.js` *18*

Módulo: `mobile/App.js` *21*

Módulo: `mobile/components/Keyboard.js` *21*

Módulo: `mobile/components/HangmanDrawing.js` *23*

Módulo: `mobile/hooks/useSocket.js` *23*

Módulo: `mobile/data/words.js` *24*

Módulo: `mobile/app/GameScreen.js` *25*

Módulo: `mobile/app/HomeScreen.js` *32*

Módulo: `mobile/app/WaitingScreen.js` *34*

6. Modelos de Datos 35

Esquema General Inferido *35*

7. Guía de Despliegue y Mantenimiento 35

7.1 Requisitos del Sistema *35*

7.2 Instalación Local *35*

7.3 Despliegue con Docker *36*

7.4 Variables de Entorno Recomendadas *36*

7.5 Checklist de Despliegue *36*

7.6 Registro de Cambios (Changelog) *36*

8. Reporte de Calidad del Código 37

8.1 Análisis de Métricas *37*

8.2 Funciones con Alta Complejidad Ciclomática *37*

8.3 Recomendaciones *37*

8.4 Errores de Análisis *37*

9. Análisis de Datos del Proyecto 37

9.1 Tipos y Fuentes de Datos Detectados 37

9.2 Esquema de Data Warehouse Inferido 38

9.3 Reporte de Calidad de Datos 38

9.4 Minería de Datos Aplicada al Código 38

9.5 Pipeline ETL Utilizado 40

juego

Documentación Técnica Exhaustiva · AutoDocs AI Enterprise · 16/10/2026

Documentación del Proyecto

*Generado automáticamente por **AutoDocs AI** — 2026-10-16 02:11 UTC*

Resumen Ejecutivo

Este proyecto está desarrollado principalmente en **JavaScript** y cuenta con 21 archivos fuente, 42 funciones, 0 clases y 3 endpoints de API documentados. El score de calidad del código es **71/100**.

Stack Tecnológico

Lenguaje	Cantidad
JavaScript	14 archivos
SQL	1 archivos
JSON	6 archivos

Estadísticas del Proyecto

Métrica	Valor
Total de archivos	21
Funciones	42
Clases	0
Endpoints API	3
Score de calidad	71/100
Funciones documentadas	0 / 42
Clases documentadas	0 / 0

2. Documentación de Arquitectura y Diseño

2.1 Estructura de Directorios

```
proyecto/
■■■ (raíz)/
■■■ backend/
■■■ mobile/
```

2.2 Distribución por Capas

- **`(raíz)/`** — 1 archivos [.gitignore(1)]

> *Módulo funcional **`(raíz)`**; contiene componentes fuente especializados.*

- **`backend/`** — 9 archivos [.js(3), .env(1), .sql(1), .sin ext(1), .json(2), .example(1)]

> *Núcleo del servidor; aloja rutas, controladores, servicios y configuración del sistema.*

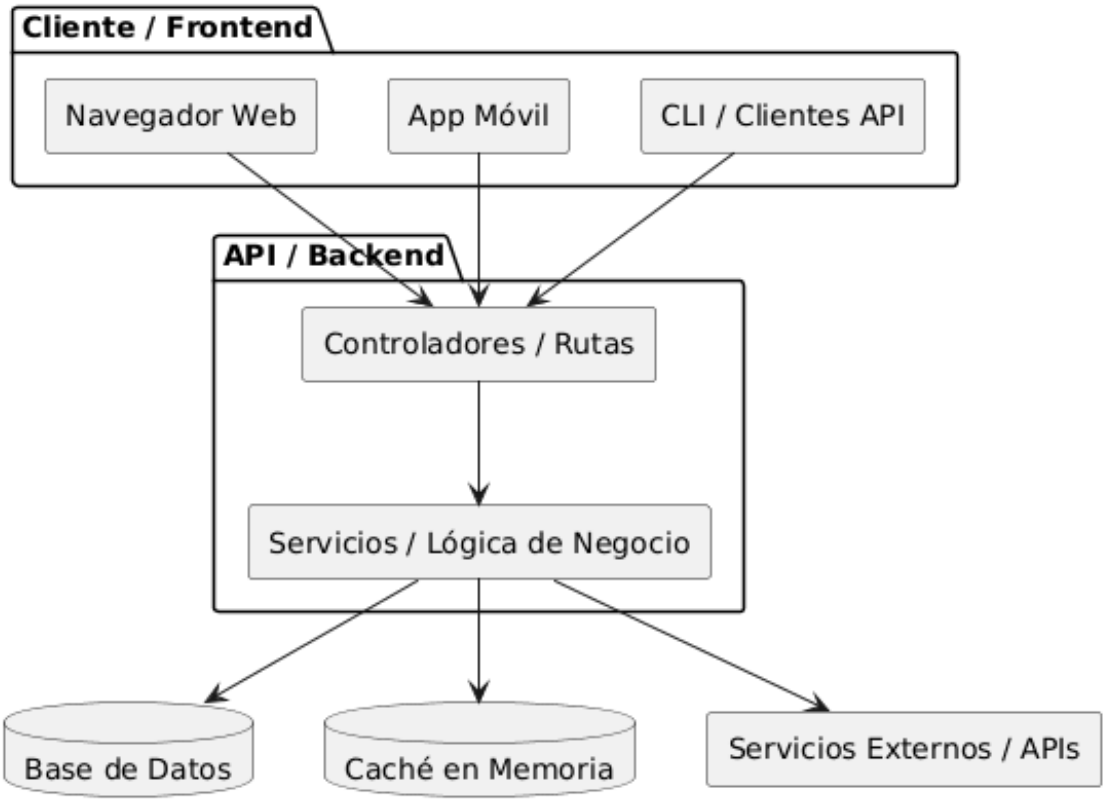
- **`mobile/`** — 15 archivos [.js(11), .json(4)]

> *Módulo funcional **`mobile`**; contiene componentes fuente especializados.*

2.3 Distribución de Lenguajes

- **JavaScript:** 14 archivos (67%)
- **JSON:** 6 archivos (29%)
- **SQL:** 1 archivos (5%)

2.4 Diagrama de Infraestructura



Descripción de la Arquitectura de Infraestructura:

El diagrama de infraestructura detalla la topología técnica y las fronteras de comunicación del sistema:

- **Capa Cliente (Frontend / Consumidores):** Comprende navegadores web, clientes móviles y scripts de consola (CLI) que se comunican mediante peticiones HTTP/REST estructuradas.
- **Capa de Transporte y API (Backend):** Módulos de enrutamiento y middleware que autentican tokens, validan parámetros de entrada y dirigen el flujo hacia los servicios.
- **Capa de Lógica de Negocio y Servicios:** Núcleo de la aplicación donde se ejecutan los algoritmos de negocio, validaciones complejas y la coordinación de datos.
- **Capa de Almacenamiento y Persistencia:** Bases de datos relacionales o NoSQL para almacenamiento duradero, combinadas con sistemas de caché en memoria para optimizar lecturas frecuentes.
- **Capa de Integraciones Externas:** Puntos de conexión con servicios de terceros (APIs de pago, mensajería, autenticación OAuth y servicios en la nube).

2.5 Diagrama de Casos de Uso

Descripción del Modelo de Casos de Uso:

Mapea las capacidades operativas provistas por el sistema y su interacción con los actores:

- **Actor (Usuario / Consumidor):** Representa al usuario autenticado o cliente de API que consume los servicios de la plataforma.

• **Casos de Uso Principales:** Cada nodo del diagrama corresponde a una funcionalidad sustantiva del sistema (autenticación, gestión de proyectos, análisis estático, consultas y reportes).

• **Flujo de Interacción:** Las peticiones del usuario disparan casos de uso atómicos, garantizando trazabilidad, seguridad y respuestas normalizadas.

2.6 Diagrama de Clases

Descripción del Diagrama de Clases del Dominio:

Representa la jerarquía orientada a objetos y las entidades estructurales detectadas en el proyecto:

• **Entidades de Dominio:** Clases que modelan los datos centrales del negocio y encapsulan las operaciones clave.

• **Cohesión y Encapsulamiento:** Cada clase organiza métodos específicos para mantener la responsabilidad única y desacoplar componentes.

• **Relaciones e Interacción:** Detalla las dependencias y asociaciones estructurales para facilitar el mantenimiento y la extensibilidad del sistema.

2.7 Patrones Detectados

• **API REST**: 3 endpoints detectados (Express.js)

• **Testing**: 1 funciones de prueba detectadas

• **Programación Asíncrona**: 12 funciones async/await

2.8 Insights de Arquitectura (IA)

No se pudieron generar insights de arquitectura

---

3. Documentación Técnica de Código

3.1 Guía de Configuración del Entorno de Desarrollo

```
# 1. Clonar el repositorio
git clone <url-del-repositorio>
cd <nombre-del-proyecto>

# 2. Crear entorno virtual (recomendado para Python)
python -m venv venv
source venv/bin/activate  # Linux/Mac
venv\Scripts\activate     # Windows

# 3. Instalar dependencias
npm install

# 4. Configurar variables de entorno
cp .env.example .env
# Editar .env con tus valores locales

# 5. Ejecutar en modo desarrollo
```

```
python app.py  # o npm run dev
```

3.2 Documentación de API

Total de endpoints detectados: **3**

Módulo: `backend/server.js`

[GET] `GET /health`

- **Framework:** Express.js
- **Archivo:** `backend/server.js`

Descripción funcional:

> *Endpoint GET /health*

Parámetros esperados:

- `Authorization` (Header, condicional): Bearer JWT token para autenticación de usuario.

Códigos de respuesta esperados:

- `200` — Operación procesada con éxito para `GET /health`.
- `400` — Solicitud incorrecta; datos o parámetros con formato no válido.
- `401` — No autorizado; falta token de acceso o credenciales expiradas.
- `404` — Recurso no encontrado en el servidor.
- `500` — Error interno en el procesamiento del servidor.

[POST] `POST /rooms`

- **Framework:** Express.js
- **Archivo:** `backend/server.js`

Descripción funcional:

> *Endpoint POST /rooms*

Parámetros esperados:

- `body` (JSON, requerido): Cuerpo de la solicitud con los datos a procesar.
- `Authorization` (Header, condicional): Bearer JWT token para autenticación de usuario.

Códigos de respuesta esperados:

- `201` — Recurso creado exitosamente para `POST /rooms`.
- `400` — Solicitud incorrecta; datos o parámetros con formato no válido.
- `401` — No autorizado; falta token de acceso o credenciales expiradas.
- `404` — Recurso no encontrado en el servidor.

- `500` — Error interno en el procesamiento del servidor.

[POST] `POST /rooms/:code/join`

- **Framework:** Express.js
- **Archivo:** `backend/server.js`

Descripción funcional:

> *Aquí tienes la explicación de esta funcionalidad, desglosada para que cualquier persona pueda entenderla sin necesidad de saber programar.*

1. ¿Cuál es su propósito en lenguaje simple?

Imagina que estás en una sala de espera de un hospital o en una sala de reuniones de una oficina. Para entrar en esa sala específica, no basta con llegar al edificio; necesitas una **llave** o un **código de acceso** que te permita abrir la puerta y sentarte dentro.

Este "punto de contacto" (lo que técnicamente llamamos endpoint) es simplemente **la puerta de entrada a esa sala**. Su único trabajo es verificar que tienes la llave correcta y, si es así, dejarte pasar al interior.

2. ¿Qué hace este endpoint?

En términos muy básicos, esta función hace dos cosas:

1. **Revisa tu invitación:** Mira el código que le enviaste para confirmar que es válido y que la sala realmente existe.

2. **Te deja entrar:** Si el código es correcto, "abre la puerta" y registra tu presencia dentro de esa sala. A partir de ese momento, formas parte de ese grupo específico y puedes interactuar con lo que haya dentro (como conversar o ver contenido).

Es como cuando entras a un grupo de WhatsApp usando un enlace de invitación: el sistema revisa el enlace y si es bueno, te añade al grupo

Parámetros esperados:

- `code` (Path, requerido): Parámetro identificador en la URL del recurso.
- `body` (JSON, requerido): Cuerpo de la solicitud con los datos a procesar.
- `Authorization` (Header, condicional): Bearer JWT token para autenticación de usuario.

Códigos de respuesta esperados:

- `201` — Recurso creado exitosamente para `POST /rooms/:code/join`.
- `400` — Solicitud incorrecta; datos o parámetros con formato no válido.
- `401` — No autorizado; falta token de acceso o credenciales expiradas.
- `404` — Recurso no encontrado en el servidor.
- `500` — Error interno en el procesamiento del servidor.

3.3 Estado de Comentarios y Docstrings

Total de funciones: 42

- **[OK] Documentadas:** 0 (0%)
- **[X] Sin documentar:** 42 (100%)

Visualización de documentación:

```
[                    ]
0% documentado
```

Funciones que requieren documentación:

- `generateRoomCode()` en `backend/server.js`
- `getRoomByCode(code)` en `backend/server.js`
- `getUsedWords(roomId)` en `backend/server.js`
- `maskWord(word, guessedLetters)` en `backend/server.js`
- `checkWin(word, guessedLetters)` en `backend/server.js`
- ... y 37 más

3.4 Planes de Prueba (Testing)

Se detectaron **1 funciones de prueba**:

Función de Test	Archivo
`testConnection`	`backend/test.js`

---

4. Clases

No se detectaron clases en este proyecto.

---

5. Documentación de Funciones

Total: **42 funciones**

Módulo: `backend/server.js`

`generateRoomCode`

- Parámetros: –
- Async: —
- Complejidad ciclomática: [LOW] 1

- Documentada: [X]

Descripción técnica:

> *La función `generateRoomCode` crea un código único e identificable para cada sala virtual. No requiere datos externos; genera automáticamente una combinación aleatoria de caracteres o números. Su resultado es un código breve y fácil de compartir, similar a una contraseña de Wi-Fi. En la práctica, permite que los usuarios se unan a reuniones o espacios de chat específicos sin necesidad de registros complejos. Es fundamental porque actúa como la "puerta de entrada" controlada: sin este código, no habría forma de distinguir entre diferentes sesiones ni de garantizar que solo las personas autorizadas accedan a cada espacio interactivo, manteniendo el orden y la privacidad dentro del sistema.*

`async getRoomByCode`

- Parámetros: `code`
- Async: [ASYNC]
- Complejidad ciclomática: [LOW] 1
- Documentada: [X]

Descripción técnica:

> *La función `getRoomByCode` actúa como un buscador específico dentro de la aplicación. Su propósito es localizar una sala o habitación concreta basándose en una etiqueta única que la identifica.*

Para funcionar, necesita únicamente ese identificador único, conocido como "código". Al recibirlo, el sistema consulta su registro interno para encontrar el espacio correspondiente.

El resultado es la información completa de esa sala, como su nombre, capacidad o características, devuelta al usuario o a otra parte del sistema que la requiera.

En la vida real, es similar a buscar un hotel por su código de confirmación o encontrar una sala de reuniones mediante su número. Sirve para que los usuarios accedan rápidamente a los detalles de un lugar específico sin tener que revisar toda la lista disponible.

Su importancia radica en permitir el acceso directo y eficiente a los recursos del sistema. Sin ella, sería imposible gestionar o visualizar una sala individual de manera precisa, lo que complicaría la experiencia del usuario y la organización interna de la plataforma.

Parámetros:

- `code`: Parámetro de entrada procesado por `getRoomByCode`.

`async getUsedWords`

- Parámetros: `roomId`
- Async: [ASYNC]
- Complejidad ciclomática: [LOW] 1
- Documentada: [X]

Descripción técnica:

> ***getUsedWords*** *actúa como un contador de palabras utilizado en una sala específica.*

1. **Propósito:** Registra y cuenta cuántas veces se han empleado ciertas palabras dentro de una conversación o sesión concreta.

2. **Datos requeridos:** Necesita saber la "identificación" única de la sala donde ocurre la interacción (roomId), similar a una clave de acceso para un grupo privado.

3. **Resultado:** Devuelve una lista o registro de las palabras que se han usado en esa sala, posiblemente con su frecuencia de aparición.

4. **Uso práctico:** Es útil para analizar tendencias de conversación, detectar temas recurrentes o medir la participación lingüística en foros, chats o salas de debate.

5. **Importancia:** Permite al sistema comprender el contenido de las interacciones, facilitando funciones de moderación, análisis de sentimiento o generación de resumos automáticos.

Al estar integrada en el servidor, garantiza que este análisis sea consistente y disponible para otras partes de la aplicación que necesiten datos sobre el lenguaje utilizado.

Parámetros:

- `roomId`: Parámetro de entrada procesado por `getUsedWords`.

`maskWord`

- Parámetros: `word, guessedLetters`
- Async: —
- Complejidad ciclomática: [LOW] 1
- Documentada: [X]

Descripción técnica:

> ***Propósito:*** *Revela parcialmente la palabra secreta en un juego de adivinanzas, mostrando solo las letras que ya se han intentado correctamente.*

Datos de entrada: Necesita la palabra completa oculta y la lista de letras que el jugador ha adivinado con éxito.

Resultado: Devuelve una versión "mascarada" de la palabra (por ejemplo, "p_l_a" en lugar de "palabra"), donde las letras correctas son visibles y las restantes aparecen como espacios o símbolos ocultos.

Uso práctico: Es la lógica central de juegos tipo "Ahorcado" o "Wordle", permitiendo al jugador ver su progreso sin revelar la solución completa.

Importancia: Es crucial para la experiencia de usuario, ya que transforma el estado interno del juego en información visualmente comprensible, manteniendo el desafío activo al ocultar los errores.

Parámetros:

- `word`: Parámetro de entrada procesado por `maskWord`.
- `guessedLetters`: Parámetro de entrada procesado por `maskWord`.

`checkWin`

- Parámetros: `word, guessedLetters`
- Async: —
- Complejidad ciclomática: [LOW] 1
- Documentada: [X]

Descripción técnica:

> `checkWin` *verifica si el jugador ha adivinado correctamente todas las letras de la palabra oculta. Necesita dos datos: la palabra objetivo y la lista de letras que el usuario ya ha intentado. Su resultado es una confirmación simple: "sí" o "no", indicando si el juego está terminado con éxito. En la práctica, esta función actúa como el árbitro final del juego; cada vez que el jugador introduce una nueva letra, el sistema consulta esta regla para decidir si mostrar un mensaje de victoria o permitir seguir intentando. Es crucial para el flujo del juego porque define el momento exacto de finalización, asegurando que el usuario solo reciba el premio o la siguiente pantalla cuando haya ganado legítimamente, manteniendo así la integridad y la lógica coherente de la experiencia de usuario.*

Parámetros:

- `word`: Parámetro de entrada procesado por `checkWin`.
- `guessedLetters`: Parámetro de entrada procesado por `checkWin`.

`serializeRoom`

- Parámetros: `room`
- Async: —
- Complejidad ciclomática: [LOW] 1
- Documentada: [X]

Descripción técnica:

> *Función serializeRoom*

Parámetros:

- `room`: Parámetro de entrada procesado por `serializeRoom`.

`async startRound`

- Parámetros: `roomCode`
- Async: [ASYNC]
- Complejidad ciclomática: [LOW] 1
- Documentada: [X]

Descripción técnica:

> *La función `startRound` actúa como el botón de "comenzar" para una nueva ronda de juego. Su propósito es iniciar la actividad dentro de una sala específica, preparando el terreno para que los participantes interactúen.*

Para funcionar, necesita un único dato: el código de la sala (`roomCode`), que es como la dirección única de esa habitación virtual. Al recibir este código, el sistema identifica el grupo de usuarios correcto.

El resultado principal es el estado activo de la ronda; el juego deja de estar en espera y comienza oficialmente. En la vida real, esto es similar a dar inicio a una partida de cartas o a una lección de clase online.

Su importancia es crítica: sin esta función, los usuarios podrían estar conectados y ver la sala, pero nunca podrían comenzar a jugar o interactuar. Es el disparador fundamental que activa la experiencia del usuario.

Parámetros:

- `roomCode`: Parámetro de entrada procesado por `startRound`.

`generateRoomCode()`

Ubicación: `backend/server.js`

Descripción detallada:

> *La función `generateRoomCode` crea un código único e identificable para cada sala virtual. No requiere datos externos; genera automáticamente una combinación aleatoria de caracteres o números. Su resultado es un código breve y fácil de compartir, similar a una contraseña de Wi-Fi. En la práctica, permite que los usuarios se unan a reuniones o espacios de chat específicos sin necesidad de registros complejos. Es fundamental porque actúa como la "puerta de entrada" controlada: sin este código, no habría forma de distinguir entre diferentes sesiones ni de garantizar que solo las personas autorizadas accedan a cada espacio interactivo, manteniendo el orden y la privacidad dentro del sistema.*

`getRoomByCode(code)`

Ubicación: `backend/server.js`

Descripción detallada:

> *La función `getRoomByCode` actúa como un buscador específico dentro de la aplicación. Su propósito es localizar una sala o habitación concreta basándose en una etiqueta única que la identifica.*

Para funcionar, necesita únicamente ese identificador único, conocido como "código". Al recibirlo, el sistema consulta su registro interno para encontrar el espacio correspondiente.

El resultado es la información completa de esa sala, como su nombre, capacidad o características, devuelta al usuario o a otra parte del sistema que la requiera.

En la vida real, es similar a buscar un hotel por su código de confirmación o encontrar una sala de reuniones mediante su número. Sirve para que los usuarios accedan rápidamente a los detalles de un lugar específico sin tener que revisar toda la lista disponible.

Su importancia radica en permitir el acceso directo y eficiente a los recursos del sistema. Sin ella, sería imposible gestionar o visualizar una sala individual de manera precisa, lo que complicaría la experiencia del usuario y la organización interna de la plataforma.

`getUsedWords(roomId)`

Ubicación: `backend/server.js`

Descripción detallada:

> ***getUsedWords*** *actúa como un contador de palabras utilizado en una sala específica.*

1. **Propósito:** Registra y cuenta cuántas veces se han empleado ciertas palabras dentro de una conversación o sesión concreta.

2. **Datos requeridos:** Necesita saber la "identificación" única de la sala donde ocurre la interacción (roomId), similar a una clave de acceso para un grupo privado.

3. **Resultado:** Devuelve una lista o registro de las palabras que se han usado en esa sala, posiblemente con su frecuencia de aparición.

4. **Uso práctico:** Es útil para analizar tendencias de conversación, detectar temas recurrentes o medir la participación lingüística en foros, chats o salas de debate.

5. **Importancia:** Permite al sistema comprender el contenido de las interacciones, facilitando funciones de moderación, análisis de sentimiento o generación de resumos automáticos.

Al estar integrada en el servidor, garantiza que este análisis sea consistente y disponible para otras partes de la aplicación que necesiten datos sobre el lenguaje utilizado.

`maskWord(word, guessedLetters)`

Ubicación: `backend/server.js`

Descripción detallada:

> ***Propósito:*** *Revela parcialmente la palabra secreta en un juego de adivinanzas, mostrando solo las letras que ya se han intentado correctamente.*

Datos de entrada: Necesita la palabra completa oculta y la lista de letras que el jugador ha adivinado con éxito.

Resultado: Devuelve una versión "mascarada" de la palabra (por ejemplo, "p_l_a" en lugar de "palabra"), donde las letras correctas son visibles y las restantes aparecen como espacios o símbolos ocultos.

Uso práctico: Es la lógica central de juegos tipo "Ahorcado" o "Wordle", permitiendo al jugador ver su progreso sin revelar la solución completa.

Importancia: Es crucial para la experiencia de usuario, ya que transforma el estado interno del juego en información visualmente comprensible, manteniendo el desafío activo al ocultar los errores.

`checkWin(word, guessedLetters)`

Ubicación: `backend/server.js`

Descripción detallada:

> *`checkWin` verifica si el jugador ha adivinado correctamente todas las letras de la palabra oculta. Necesita dos datos: la palabra objetivo y la lista de letras que el usuario ya ha intentado. Su resultado es una confirmación simple: "sí" o "no", indicando si el juego está terminado con éxito. En la práctica, esta función actúa como el árbitro final del juego; cada vez que el jugador introduce una nueva letra, el sistema consulta esta regla para decidir si mostrar un mensaje de victoria o permitir seguir intentando. Es crucial para el flujo del juego porque define el momento exacto de finalización, asegurando que el usuario solo reciba el premio o la siguiente pantalla cuando haya ganado legítimamente, manteniendo así la integridad y la lógica coherente de la experiencia de usuario.*

`serializeRoom(room)`

Ubicación: `backend/server.js`

Descripción detallada:

> *Función serializeRoom*

`startRound(roomCode)`

Ubicación: `backend/server.js`

Descripción detallada:

> *La función `startRound` actúa como el botón de "comenzar" para una nueva ronda de juego. Su propósito es iniciar la actividad dentro de una sala específica, preparando el terreno para que los participantes interactúen.*

Para funcionar, necesita un único dato: el código de la sala (`roomCode`), que es como la dirección única de esa habitación virtual. Al recibir este código, el sistema identifica el grupo de usuarios correcto.

El resultado principal es el estado activo de la ronda; el juego deja de estar en espera y comienza oficialmente. En la vida real, esto es similar a dar inicio a una partida de cartas o a una lección de clase online.

Su importancia es crítica: sin esta función, los usuarios podrían estar conectados y ver la sala, pero nunca podrían comenzar a jugar o interactuar. Es el disparador fundamental que activa la experiencia del usuario.

Módulo: `backend/test.js`

`async testConnection`

- Parámetros: —
- Async: [ASYNC]
- Complejidad ciclomática: [LOW] 1
- Documentada: [X]

Descripción técnica:

> *Función testConnection*

`testConnection()`

Ubicación: `backend/test.js`

Descripción detallada:

> *Función testConnection*

Módulo: `backend/words.js`

`isValidWord`

- Parámetros: `word`
- Async: —
- Complejidad ciclomática: [LOW] 1
- Documentada: [X]

Descripción técnica:

> *Función isValidWord*

Parámetros:

- `word`: Parámetro de entrada procesado por `isValidWord`.

`normalizeWord`

- Parámetros: `word`
- Async: —
- Complejidad ciclomática: [LOW] 1
- Documentada: [X]

Descripción técnica:

> *Función normalizeWord*

Parámetros:

- `word`: Parámetro de entrada procesado por `normalizeWord`.

`async fetchFromPalabrasDev`

- Parámetros: —
- Async: [ASYNC]
- Complejidad ciclomática: [LOW] 1
- Documentada: [X]

Descripción técnica:

> *Función fetchFromPalabrasDev*

`async fetchBatchFromPalabrasDev`

- Parámetros: `count = 5`
- Async: [ASYNC]
- Complejidad ciclomática: [LOW] 1
- Documentada: [X]

Descripción técnica:

> ***fetchBatchFromPalabrasDev***

1. **Propósito**: Esta función actúa como un "buscador de lotes". Su tarea principal es solicitar un grupo específico de palabras de una fuente de datos de desarrollo, permitiendo manejarlas en bloques organizados en lugar de una por una.

2. **Datos necesarios**: Solo requiere saber la cantidad de elementos que se desean recuperar. Por defecto, pide 5 palabras, aunque este número se puede ajustar según las necesidades del momento.

3. **Resultado**: Devuelve una lista concreta con las palabras solicitadas, lista para ser procesada o mostrada inmediatamente.

4. **Uso práctico**: Es útil para cargar contenido de forma progresiva. Por ejemplo, al abrir una aplicación de memoria, esta función permite cargar las primeras cinco tarjetas de palabras para que la interfaz no se frene mientras carga el resto.

5. **Importancia**: Optimiza el rendimiento del sistema y mejora la experiencia del usuario al dividir grandes volúmenes de información en porciones manejables y rápidas de cargar.

Parámetros:

- `count = 5`: Parámetro de entrada procesado por `fetchBatchFromPalabrasDev`.

`async getRandomWord`

- Parámetros: `usedWords = []`
- Async: [ASYNC]
- Complejidad ciclomática: [LOW] 1

- Documentada: [X]

Descripción técnica:

> *Función getRandomWord*

Parámetros:

- **`usedWords = []`**: Parámetro de entrada procesado por **`getRandomWord`**.

`isValidWord(word)`

Ubicación: **`backend/words.js`**

Descripción detallada:

> *Función isValidWord*

`normalizeWord(word)`

Ubicación: **`backend/words.js`**

Descripción detallada:

> *Función normalizeWord*

`fetchFromPalabrasDev()`

Ubicación: **`backend/words.js`**

Descripción detallada:

> *Función fetchFromPalabrasDev*

`fetchBatchFromPalabrasDev(count = 5)`

Ubicación: **`backend/words.js`**

Descripción detallada:

> ***fetchBatchFromPalabrasDev***

1. **Propósito**: Esta función actúa como un "buscador de lotes". Su tarea principal es solicitar un grupo específico de palabras de una fuente de datos de desarrollo, permitiendo manejarlas en bloques organizados en lugar de una por una.

2. **Datos necesarios**: Solo requiere saber la cantidad de elementos que se desean recuperar. Por defecto, pide 5 palabras, aunque este número se puede ajustar según las necesidades del momento.

3. **Resultado**: Devuelve una lista concreta con las palabras solicitadas, lista para ser procesada o mostrada inmediatamente.

4. **Uso práctico**: Es útil para cargar contenido de forma progresiva. Por ejemplo, al abrir una aplicación de memoria, esta función permite cargar las primeras cinco tarjetas de palabras para que la interfaz no se frene mientras carga el resto.

5. **Importancia**: Optimiza el rendimiento del sistema y mejora la experiencia del usuario al dividir grandes volúmenes de información en porciones manejables y rápidas de cargar.

`getRandomWord(usedWords = [])`

Ubicación: `backend/words.js`

Descripción detallada:

> *Función getRandomWord*

Módulo: `mobile/App.js`

`App`

- Parámetros: `—`
- Async: —
- Complejidad ciclomática: [LOW] 1
- Documentada: [X]

Descripción técnica:

> *Función App*

`App()`

Ubicación: `mobile/App.js`

Descripción detallada:

> *Función App*

Módulo: `mobile/components/Keyboard.js`

`Keyboard`

- Parámetros: `{ guessedLetters, onGuess, disabled, currentWord }`
- Async: —
- Complejidad ciclomática: [LOW] 1
- Documentada: [X]

Descripción técnica:

> *Función Keyboard*

Parámetros:

- `{ guessedLetters`: Parámetro de entrada procesado por `Keyboard`.
- `onGuess`: Parámetro de entrada procesado por `Keyboard`.
- `disabled`: Parámetro de entrada procesado por `Keyboard`.
- `currentWord }`: Parámetro de entrada procesado por `Keyboard`.

`getLetterStatus`

- Parámetros: `letter`
- Async: —
- Complejidad ciclomática: [LOW] 1
- Documentada: [X]

Descripción técnica:

> *Ejecuta la funcionalidad `getLetterStatus` sobre los datos del contexto, procesando los parámetros (letter).*

Parámetros:

- `letter`: Parámetro de entrada procesado por `getLetterStatus`.

`renderKey`

- Parámetros: `letter`
- Async: —
- Complejidad ciclomática: [LOW] 1
- Documentada: [X]

Descripción técnica:

> *Ejecuta la funcionalidad `renderKey` sobre los datos del contexto, procesando los parámetros (letter).*

Parámetros:

- `letter`: Parámetro de entrada procesado por `renderKey`.

`Keyboard({ guessedLetters, onGuess, disabled, currentWord })`

Ubicación: `mobile/components/Keyboard.js`

Descripción detallada:

> *Función Keyboard*

`getLetterStatus(letter)`

Ubicación: `mobile/components/Keyboard.js`

Descripción detallada:

> *Ejecuta la funcionalidad `getLetterStatus` sobre los datos del contexto, procesando los parámetros (letter).*

`renderKey(letter)`

Ubicación: `mobile/components/Keyboard.js`

Descripción detallada:

> *Ejecuta la funcionalidad `renderKey` sobre los datos del contexto, procesando los parámetros (letter).*

Módulo: `mobile/components/HangmanDrawing.js`

`HangmanDrawing`

- Parámetros: `{ wrongAttempts, size = 200 }`
- Async: —
- Complejidad ciclomática: [LOW] 1
- Documentada: [X]

Descripción técnica:

> *Ejecuta la funcionalidad `HangmanDrawing` sobre los datos del contexto, procesando los parámetros ({ wrongAttempts, size = 200 }).*

Parámetros:

- `{ wrongAttempts`: Parámetro de entrada procesado por `HangmanDrawing`.
- `size = 200 }`: Parámetro de entrada procesado por `HangmanDrawing`.

`HangmanDrawing({ wrongAttempts, size = 200 })`

Ubicación: `mobile/components/HangmanDrawing.js`

Descripción detallada:

> *Ejecuta la funcionalidad `HangmanDrawing` sobre los datos del contexto, procesando los parámetros ({ wrongAttempts, size = 200 }).*

Módulo: `mobile/hooks/useSocket.js`

`useSocket`

- Parámetros: –
- Async: —
- Complejidad ciclomática: [LOW] 1
- Documentada: [X]

Descripción técnica:

> *Ejecuta la funcionalidad* `useSocket` *sobre los datos del contexto sin requerir parámetros obligatorios.*

`useSocket()`

Ubicación: `mobile/hooks/useSocket.js`

Descripción detallada:

> *Ejecuta la funcionalidad* `useSocket` *sobre los datos del contexto sin requerir parámetros obligatorios.*

Módulo: `mobile/data/words.js`

`async getRandomWord`

- Parámetros: `usedWords = []`
- Async: [ASYNC]
- Complejidad ciclomática: [LOW] 1
- Documentada: [X]

Descripción técnica:

> *Ejecuta la funcionalidad* `getRandomWord` *sobre los datos del contexto de forma asíncrona, procesando los parámetros (usedWords = []).*

Parámetros:

- `usedWords = []`: Parámetro de entrada procesado por `getRandomWord`.

`getRandomWordFallback`

- Parámetros: `usedWords = []`
- Async: —
- Complejidad ciclomática: [LOW] 1
- Documentada: [X]

Descripción técnica:

> *Ejecuta la funcionalidad* `getRandomWordFallback` *sobre los datos del contexto, procesando los parámetros (usedWords = []).*

Parámetros:

- `usedWords = []`: Parámetro de entrada procesado por `getRandomWordFallback`.

`getRandomWord(usedWords = [])`

Ubicación: `mobile/data/words.js`

Descripción detallada:

> *Ejecuta la funcionalidad `getRandomWord` sobre los datos del contexto de forma asíncrona, procesando los parámetros (usedWords = []).*

`getRandomWordFallback(usedWords = [])`

Ubicación: `mobile/data/words.js`

Descripción detallada:

> *Ejecuta la funcionalidad `getRandomWordFallback` sobre los datos del contexto, procesando los parámetros (usedWords = []).*

Módulo: `mobile/app/GameScreen.js`

`useSounds`

- Parámetros: –
- Async: —
- Complejidad ciclomática: [LOW] 1
- Documentada: [X]

Descripción técnica:

> *Ejecuta la funcionalidad `useSounds` sobre los datos del contexto sin requerir parámetros obligatorios.*

`buildInitialSPState`

- Parámetros: `{ word, playerName, difficulty, hintsCount, playerWins }`
- Async: —
- Complejidad ciclomática: [LOW] 1
- Documentada: [X]

Descripción técnica:

> *Ejecuta la funcionalidad `buildInitialSPState` sobre los datos del contexto, procesando los parámetros ({ word, playerName, difficulty, hintsCount, playerWins }).*

Parámetros:

- `{ word`: Parámetro de entrada procesado por `buildInitialSPState`.

- `playerName`: Parámetro de entrada procesado por `buildInitialSPState`.
- `difficulty`: Parámetro de entrada procesado por `buildInitialSPState`.
- `hintsCount`: Parámetro de entrada procesado por `buildInitialSPState`.
- `playerWins }`: Parámetro de entrada procesado por `buildInitialSPState`.

`GameScreen`

- Parámetros: `{ route, navigation }`
- Async: —
- Complejidad ciclomática: [LOW] 1
- Documentada: [X]

Descripción técnica:

> *Ejecuta la funcionalidad `GameScreen` sobre los datos del contexto, procesando los parámetros ({ route, navigation }).*

Parámetros:

- `{ route`: Parámetro de entrada procesado por `GameScreen`.
- `navigation }`: Parámetro de entrada procesado por `GameScreen`.

`clearTimerInterval`

- Parámetros: `—`
- Async: —
- Complejidad ciclomática: [LOW] 1
- Documentada: [X]

Descripción técnica:

> *Ejecuta la funcionalidad `clearTimerInterval` sobre los datos del contexto sin requerir parámetros obligatorios.*

`resetTimer`

- Parámetros: `—`
- Async: —
- Complejidad ciclomática: [LOW] 1
- Documentada: [X]

Descripción técnica:

> *Ejecuta la funcionalidad `resetTimer` sobre los datos del contexto sin requerir parámetros obligatorios.*

`startTimer`

- Parámetros: —
- Async: —
- Complejidad ciclomática: [LOW] 1
- Documentada: [X]

Descripción técnica:

> *Ejecuta la funcionalidad `startTimer` sobre los datos del contexto sin requerir parámetros obligatorios.*

`StatCell`

- Parámetros: `{ icon, color, label, value }`
- Async: —
- Complejidad ciclomática: [LOW] 1
- Documentada: [X]

Descripción técnica:

> *Ejecuta la funcionalidad `StatCell` sobre los datos del contexto, procesando los parámetros ({ icon, color, label, value }).*

Parámetros:

- `{ icon`: Parámetro de entrada procesado por `StatCell`.
- `color`: Parámetro de entrada procesado por `StatCell`.
- `label`: Parámetro de entrada procesado por `StatCell`.
- `value }`: Parámetro de entrada procesado por `StatCell`.

`shakeScreen`

- Parámetros: —
- Async: —
- Complejidad ciclomática: [LOW] 1
- Documentada: [X]

Descripción técnica:

> *Ejecuta la funcionalidad `shakeScreen` sobre los datos del contexto sin requerir parámetros obligatorios.*

`bounceWin`

- Parámetros: —
- Async: —
- Complejidad ciclomática: [LOW] 1
- Documentada: [X]

Descripción técnica:

> *Ejecuta la funcionalidad `bounceWin` sobre los datos del contexto sin requerir parámetros obligatorios.*

`flashLoss`

- Parámetros: `—`
- Async: —
- Complejidad ciclomática: [LOW] 1
- Documentada: [X]

Descripción técnica:

> *Ejecuta la funcionalidad `flashLoss` sobre los datos del contexto sin requerir parámetros obligatorios.*

`pulseLetter`

- Parámetros: `—`
- Async: —
- Complejidad ciclomática: [LOW] 1
- Documentada: [X]

Descripción técnica:

> *Ejecuta la funcionalidad `pulseLetter` sobre los datos del contexto sin requerir parámetros obligatorios.*

`handleHint`

- Parámetros: `—`
- Async: —
- Complejidad ciclomática: [LOW] 1
- Documentada: [X]

Descripción técnica:

> *Ejecuta la funcionalidad `handleHint` sobre los datos del contexto sin requerir parámetros obligatorios.*

`processGuess`

- Parámetros: `letter, isHint = false`
- Async: —
- Complejidad ciclomática: [LOW] 1
- Documentada: [X]

Descripción técnica:

Ejecuta la funcionalidad `processGuess` sobre los datos del contexto, procesando los parámetros (letter, isHint = false).

Parámetros:

- `letter`: Parámetro de entrada procesado por `processGuess`.
- `isHint = false`: Parámetro de entrada procesado por `processGuess`.

`handleGuess`

- Parámetros: `letter`
- Async: —
- Complejidad ciclomática: [LOW] 1
- Documentada: [X]

Descripción técnica:

Ejecuta la funcionalidad `handleGuess` sobre los datos del contexto, procesando los parámetros (letter).

Parámetros:

- `letter`: Parámetro de entrada procesado por `handleGuess`.

`handleExit`

- Parámetros: `confirmed = false`
- Async: —
- Complejidad ciclomática: [LOW] 1
- Documentada: [X]

Descripción técnica:

Ejecuta la funcionalidad `handleExit` sobre los datos del contexto, procesando los parámetros (confirmed = false).

Parámetros:

- `confirmed = false`: Parámetro de entrada procesado por `handleExit`.

`async handleNewGame`

- Parámetros: `—`
- Async: [ASYNC]
- Complejidad ciclomática: [LOW] 1
- Documentada: [X]

Descripción técnica:

Ejecuta la funcionalidad `handleNewGame` sobre los datos del contexto de forma asíncrona sin requerir parámetros obligatorios.

`useSounds()`

Ubicación: `mobile/app/GameScreen.js`

Descripción detallada:

Ejecuta la funcionalidad `useSounds` sobre los datos del contexto sin requerir parámetros obligatorios.

`buildInitialSPState({ word, playerName, difficulty, hintsCount, playerWins })`

Ubicación: `mobile/app/GameScreen.js`

Descripción detallada:

Ejecuta la funcionalidad `buildInitialSPState` sobre los datos del contexto, procesando los parámetros ({ word, playerName, difficulty, hintsCount, playerWins }).

`GameScreen({ route, navigation })`

Ubicación: `mobile/app/GameScreen.js`

Descripción detallada:

Ejecuta la funcionalidad `GameScreen` sobre los datos del contexto, procesando los parámetros ({ route, navigation }).

`clearTimerInterval()`

Ubicación: `mobile/app/GameScreen.js`

Descripción detallada:

Ejecuta la funcionalidad `clearTimerInterval` sobre los datos del contexto sin requerir parámetros obligatorios.

`resetTimer()`

Ubicación: `mobile/app/GameScreen.js`

Descripción detallada:

Ejecuta la funcionalidad `resetTimer` sobre los datos del contexto sin requerir parámetros obligatorios.

`startTimer()`

Ubicación: `mobile/app/GameScreen.js`

Descripción detallada:

Ejecuta la funcionalidad `startTimer` sobre los datos del contexto sin requerir parámetros obligatorios.

`StatCell({ icon, color, label, value })`

Ubicación: `mobile/app/GameScreen.js`

Descripción detallada:

Ejecuta la funcionalidad `StatCell` sobre los datos del contexto, procesando los parámetros ({ icon, color, label, value }).

`shakeScreen()`

Ubicación: `mobile/app/GameScreen.js`

Descripción detallada:

Ejecuta la funcionalidad `shakeScreen` sobre los datos del contexto sin requerir parámetros obligatorios.

`bounceWin()`

Ubicación: `mobile/app/GameScreen.js`

Descripción detallada:

Ejecuta la funcionalidad `bounceWin` sobre los datos del contexto sin requerir parámetros obligatorios.

`flashLoss()`

Ubicación: `mobile/app/GameScreen.js`

Descripción detallada:

Ejecuta la funcionalidad `flashLoss` sobre los datos del contexto sin requerir parámetros obligatorios.

`pulseLetter()`

Ubicación: `mobile/app/GameScreen.js`

Descripción detallada:

Ejecuta la funcionalidad `pulseLetter` sobre los datos del contexto sin requerir parámetros obligatorios.

`handleHint()`

Ubicación: `mobile/app/GameScreen.js`

Descripción detallada:

Ejecuta la funcionalidad `handleHint` sobre los datos del contexto sin requerir parámetros obligatorios.

`processGuess(letter, isHint = false)`

Ubicación: `mobile/app/GameScreen.js`

Descripción detallada:

Ejecuta la funcionalidad `processGuess` sobre los datos del contexto, procesando los parámetros (letter, isHint = false).

`handleGuess(letter)`

Ubicación: `mobile/app/GameScreen.js`

Descripción detallada:

Ejecuta la funcionalidad `handleGuess` sobre los datos del contexto, procesando los parámetros (letter).

`handleExit(confirmed = false)`

Ubicación: `mobile/app/GameScreen.js`

Descripción detallada:

Ejecuta la funcionalidad `handleExit` sobre los datos del contexto, procesando los parámetros (confirmed = false).

`handleNewGame()`

Ubicación: `mobile/app/GameScreen.js`

Descripción detallada:

Ejecuta la funcionalidad `handleNewGame` sobre los datos del contexto de forma asíncrona sin requerir parámetros obligatorios.

Módulo: `mobile/app/HomeScreen.js`

`HomeScreen`

- Parámetros: `{ route, navigation }`
- Async: —
- Complejidad ciclomática: [LOW] 1
- Documentada: [X]

Descripción técnica:

> *Ejecuta la funcionalidad `HomeScreen` sobre los datos del contexto, procesando los parámetros ({ route, navigation }).*

Parámetros:

- `{ route`: Parámetro de entrada procesado por `HomeScreen`.
- `navigation }`: Parámetro de entrada procesado por `HomeScreen`.

`async createRoom`

- Parámetros: `—`
- Async: [ASYNC]
- Complejidad ciclomática: [LOW] 1
- Documentada: [X]

Descripción técnica:

> *Ejecuta la funcionalidad `createRoom` sobre los datos del contexto de forma asíncrona sin requerir parámetros obligatorios.*

`async joinRoom`

- Parámetros: `—`
- Async: [ASYNC]
- Complejidad ciclomática: [LOW] 1
- Documentada: [X]

Descripción técnica:

> *Ejecuta la funcionalidad `joinRoom` sobre los datos del contexto de forma asíncrona sin requerir parámetros obligatorios.*

`HomeScreen({ route, navigation })`

Ubicación: `mobile/app/HomeScreen.js`

Descripción detallada:

> *Ejecuta la funcionalidad `HomeScreen` sobre los datos del contexto, procesando los parámetros ({ route, navigation }).*

`createRoom()`

Ubicación: `mobile/app/HomeScreen.js`

Descripción detallada:

Ejecuta la funcionalidad `createRoom` sobre los datos del contexto de forma asíncrona sin requerir parámetros obligatorios.

`joinRoom()`

Ubicación: `mobile/app/HomeScreen.js`

Descripción detallada:

Ejecuta la funcionalidad `joinRoom` sobre los datos del contexto de forma asíncrona sin requerir parámetros obligatorios.

Módulo: `mobile/app/WaitingScreen.js`

`WaitingScreen`

- Parámetros: `{ route, navigation }`
- Async: —
- Complejidad ciclomática: [LOW] 1
- Documentada: [X]

Descripción técnica:

Ejecuta la funcionalidad `WaitingScreen` sobre los datos del contexto, procesando los parámetros ({ route, navigation }).

Parámetros:

- `{ route`: Parámetro de entrada procesado por `WaitingScreen`.
- `navigation }`: Parámetro de entrada procesado por `WaitingScreen`.

`async shareCode`

- Parámetros: `—`
- Async: [ASYNC]
- Complejidad ciclomática: [LOW] 1
- Documentada: [X]

Descripción técnica:

Ejecuta la funcionalidad `shareCode` sobre los datos del contexto de forma asíncrona sin requerir parámetros obligatorios.

`WaitingScreen({ route, navigation })`

Ubicación: `mobile/app/WaitingScreen.js`

Descripción detallada:

Ejecuta la funcionalidad `WaitingScreen` sobre los datos del contexto, procesando los parámetros ({ route, navigation }).

`shareCode()`

Ubicación: `mobile/app/WaitingScreen.js`

Descripción detallada:

Ejecuta la funcionalidad `shareCode` sobre los datos del contexto de forma asíncrona sin requerir parámetros obligatorios.

---

6. Modelos de Datos

No se detectaron clases con nombres de modelo explícitos.

Esquema General Inferido

Basado en el análisis de código y funciones detectadas, las entidades principales del dominio son:

- Entidades estructurales inferidas a partir de los módulos de la aplicación.

---

7. Guía de Despliegue y Mantenimiento

7.1 Requisitos del Sistema

- **Lenguaje principal**: JavaScript
- **Archivos detectados**: 21

7.2 Instalación Local

```
# 1. Clonar el repositorio
git clone <url-del-repositorio>
cd <nombre-del-proyecto>

# 2. Instalar dependencias
npm install

# 3. Configurar variables de entorno
cp .env.example .env
# Editar .env con tus valores

# 4. Ejecutar el proyecto
npm start
```

7.3 Despliegue con Docker

```
FROM node:20-slim

WORKDIR /app
COPY . .
RUN npm install
EXPOSE 8000
CMD ["node index.js"]
```

```
# Construir imagen
docker build -t mi-proyecto .

# Ejecutar contenedor
docker run -p 8000:8000 mi-proyecto
```

7.4 Variables de Entorno Recomendadas

```
# General
APP_ENV=production
SECRET_KEY=clave-secreta-segura
DEBUG=False

# Base de datos
DB_HOST=localhost
DB_PORT=5432
DB_NAME=mi_base_datos
DB_USER=usuario
DB_PASSWORD=contraseña

# API
API_VERSION=v1
ALLOWED_HOSTS=*
```

7.5 Checklist de Despliegue

- [ ] Variables de entorno configuradas
- [ ] Base de datos migrada
- [ ] Tests ejecutados y pasando
- [ ] Logs configurados
- [ ] HTTPS habilitado en producción
- [ ] Backups programados
- [ ] Monitoreo configurado

7.6 Registro de Cambios (Changelog)

Versión 1.0.0 — 2026-10-16

- Versión inicial analizada por AutoDocs AI

- Documentación técnica generada automáticamente
- Análisis de calidad del código completado

---

8. Reporte de Calidad del Código

8.1 Análisis de Métricas

Score de Calidad: 71/100 (Bueno [YELLOW])

Distribución de Documentación:

- Funciones documentadas: 0/42 (0%)
- Clases documentadas: 0/0

Análisis de Complejidad:

- Complejidad promedio: 1.0
- Complejidad máxima: 1
- Funciones con alta complejidad (>10): 0

Tendencia de Complejidad por Archivo:

```
[Gráfico de regresión lineal mostrando la tendencia de complejidad]
Eje X: Archivos ordenados por complejidad promedio
Eje Y: Score de complejidad
Tendencia: ↓ Baja
```

8.2 Funciones con Alta Complejidad Ciclomática

No hay funciones con complejidad > 10. [OK]

8.3 Recomendaciones

- Agregar docstrings a las 42 funciones sin documentar

8.4 Errores de Análisis

Sin errores detectados. [OK]

---

9. Análisis de Datos del Proyecto

9.1 Tipos y Fuentes de Datos Detectados

Datos Estructurados: 0 archivos

Datos Semi-estructurados: 0 archivos

Datos No Estructurados: 0 archivos

9.2 Esquema de Data Warehouse Inferido

No se detectaron suficientes modelos de datos para generar un esquema de Data Warehouse.

9.3 Reporte de Calidad de Datos

Problemas Detectados en el Código:

- **Funciones sin documentar:** 42

- `generateRoomCode` en `backend/server.js`
- `getRoomByCode` en `backend/server.js`
- `getUsedWords` en `backend/server.js`
- `maskWord` en `backend/server.js`
- `checkWin` en `backend/server.js`
- ... y 37 más

9.4 Minería de Datos Aplicada al Código

Análisis automático realizado sobre las funciones y clases extraídas del proyecto.

A) Funciones de Alta Complejidad Ciclomática

Nivel	Umbral	Cantidad	Descripción
■ Alta	> 5	0	Requieren refactorización urgente
■ Media	4–5	0	Candidatas a simplificación
■ Baja	≤ 3	42	Complejidad aceptable

■ *No se detectaron funciones con complejidad ciclomática alta. El código mantiene un buen nivel de simplicidad.*

B) Funciones Sin Documentación

42 de 42 funciones carecen de docstring (100%).

Función	Archivo	Línea	Complejidad
`generateRoomCode`	`backend/server.js`	?	1
`getRoomByCode`	`backend/server.js`	?	1
`getUsedWords`	`backend/server.js`	?	1
`maskWord`	`backend/server.js`	?	1

checkWin	backend/server.js	?	1
serializeRoom	backend/server.js	?	1
startRound	backend/server.js	?	1
testConnection	backend/test.js	?	1
isValidWord	backend/words.js	?	1
normalizeWord	backend/words.js	?	1
fetchFromPalabrasDev	backend/words.js	?	1
fetchBatchFromPalabrasDev	backend/words.js	?	1
getRandomWord	backend/words.js	?	1
App	mobile/App.js	?	1
Keyboard	mobile/components/Keyboard.js	?	1
getLetterStatus	mobile/components/Keyboard.js	?	1
renderKey	mobile/components/Keyboard.js	?	1
HangmanDrawing	mobile/components/HangmanDrawing.js	?	1
useSocket	mobile/hooks/useSocket.js	?	1
getRandomWord	mobile/data/words.js	?	1

... y 22 funciones más sin documentar.

■ **Sugerencia:** Agrega docstrings a todas las funciones públicas. Prioriza las de mayor complejidad primero, ya que son las más difíciles de entender sin documentación.

C) Detección de Funciones con Nombres Similares (Posibles Duplicados)

Se comparan los nombres de funciones entre archivos para detectar lógica duplicada o que podría unificarse.

Se detectaron **1 grupos de funciones con nombres similares** repartidas en distintos archivos:

Patrón: _getrandomword / getrandomword_ — aparece en 2 archivos distintos

Función	Archivo	Línea	Complejidad
getRandomWord	backend/words.js	?	1
getRandomWord	mobile/data/words.js	?	1

■ **Sugerencia:** Las funciones `getRandomWord` y `getRandomWord` realizan operaciones similares (`getrandomword`) en módulos distintos. Considera crear una función compartida en un módulo utilitario (por ejemplo `utils/getrandomword.py` o `helpers/common.py`) que reciba el contexto como parámetro, y llamarla desde ambos archivos (`backend/words.js`, `mobile/data/words.js`).

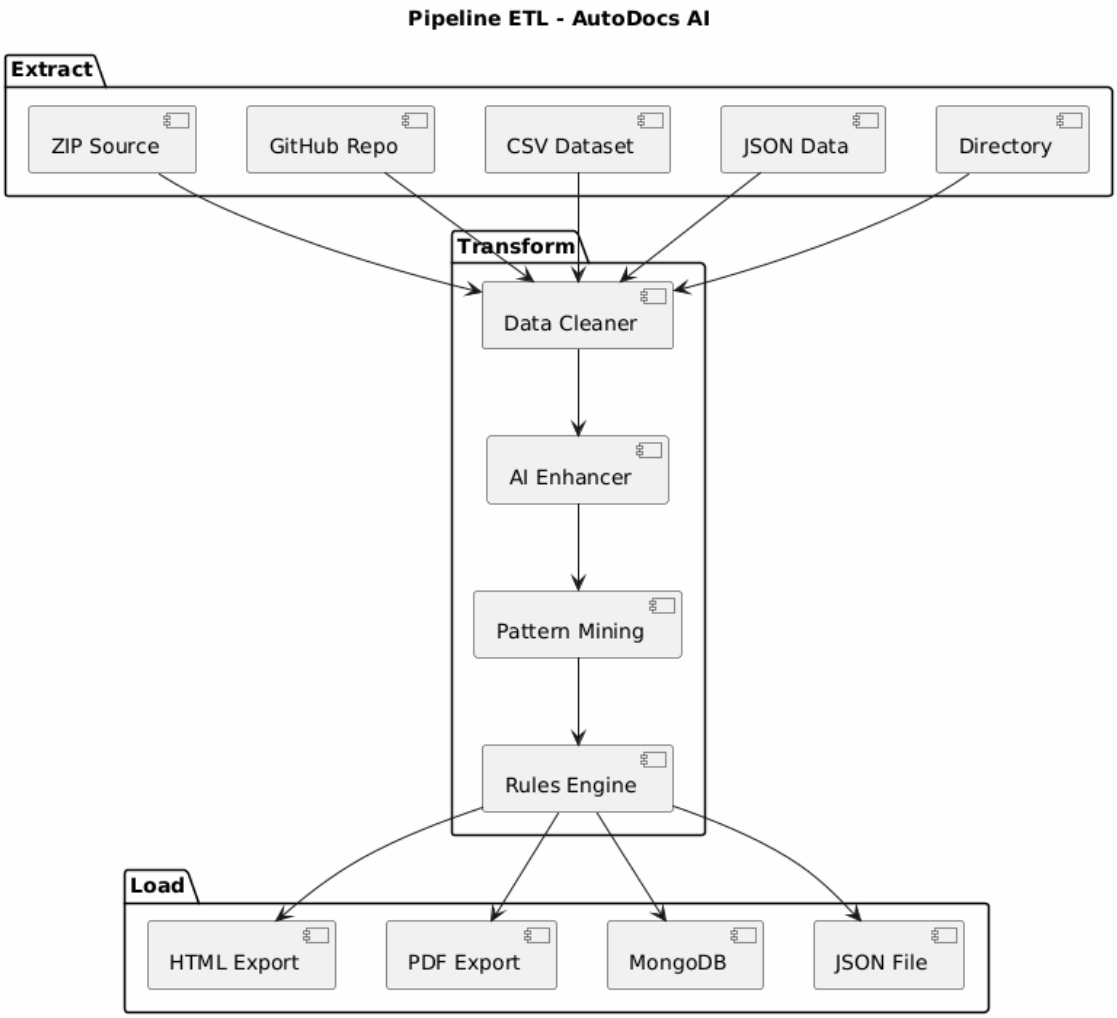
D) Análisis de Módulos — Archivos con Mayor Concentración de Funciones

Archivo	Funciones	Complejidad Promedio	Documentadas	Estado
`mobile/app/GameScreen.js`	16	1.0	0%	■ Revisar
`backend/server.js`	7	1.0	0%	■ OK
`backend/words.js`	5	1.0	0%	■ OK
`mobile/components/Keyboard.js`	3	1.0	0%	■ OK
`mobile/app/HomeScreen.js`	3	1.0	0%	■ OK
`mobile/data/words.js`	2	1.0	0%	■ OK
`mobile/app/WaitingScreen.js`	2	1.0	0%	■ OK
`backend/test.js`	1	1.0	0%	■ OK
`mobile/App.js`	1	1.0	0%	■ OK
`mobile/components/HangmanDrawing.js`	1	1.0	0%	■ OK
`mobile/hooks/useSocket.js`	1	1.0	0%	■ OK

■ `mobile/app/GameScreen.js` tiene 16 funciones. Considera dividirlo en submódulos más específicos siguiendo el principio de responsabilidad única (SRP).

9.5 Pipeline ETL Utilizado

El proceso de generación de documentación sigue un pipeline ETL (Extract, Transform, Load):



Fases del Pipeline:

- **Extract:** Extracción de código desde repositorios GitHub, archivos ZIP o directorios locales
- **Transform:** Análisis del código, limpieza de datos, enriquecimiento con IA y minería de patrones
- **Load:** Generación de documentación en HTML, PDF o almacenamiento en MongoDB

Documentación generada automáticamente por AutoDocs AI — 2026-10-16 02:11 UTC
